

Pseudocode:

```
// Input:
N = 10 temperature values (float) in Kelvin degrees
Threshold temperature T (float) in Kelvin degrees

// Output:
State of water sample (Solid, gas, liquid)
Converted temperature (Celsius and Fahrenheit)
Maximum temperature
Minimum temperature
Average temperature

// High-Level Algorithm:

BEGIN MAIN
// Function prototypes
DECLARE FUNCTION kelvinToCelsius(Kelvin)
DECLARE FUNCTION celsiusToFahrenheit(Celsius)
DECLARE FUNCTION determineState(Celsius)
DECLARE FUNCTION findMax(Array, Size)
DECLARE FUNCTION findMin(Array, Size)
DECLARE FUNCTION calculateAverage(Array, Size)
DECLARE FUNCTION countBelowThreshold(Array, Size, Threshold)

CONSTANT SIZE = 10

DECLARE KelvinTemps[SIZE], CelsiusTemps[SIZE],
FahrenheitTemps[SIZE]
```

```

// Read SIZE temperature values in Kelvin
FOR i FROM 0 TO SIZE-1 DO
    KelvinTemps[i] = READ_FLOAT("Enter temperature in Kelvin:
")
END FOR

// Convert temperatures to Celsius and Fahrenheit
FOR i FROM 0 TO SIZE-1 DO
    CelsiusTemps[i] = kelvinToCelsius(KelvinTemps[i])
    FahrenheitTemps[i] = celsiusToFahrenheit(CelsiusTemps[i])
END FOR

// Display results
PRINT "Kelvin    Celsius    Fahrenheit    State"

// Find the maximum and minimum temperatures
PRINT "Max temperature: ", findMax(KelvinTemps, SIZE)
PRINT "Min temperature: ", findMin(KelvinTemps, SIZE)

// Compute the average temperature
PRINT "Average temperature: ", calculateAverage(KelvinTemps,
SIZE)

// Count the number of samples below threshold T
DECLARE Threshold = READ_FLOAT("Enter the threshold
temperature in Kelvin: ")
DECLARE CountBelowThreshold = countBelowThreshold(KelvinTemps,
SIZE, Threshold)
PRINT "Number of samples below ", Threshold, " K: ",
CountBelowThreshold

```

```
END MAIN
```

```
// High-Level Algorithm End
```

```
// Low-Level Algorithm: Start
```

```
Step 1 Refinement
```

```
FUNCTION kelvinToCelsius(Kelvin)
```

```
    RETURN Kelvin - 273.15
```

```
END FUNCTION
```

```
Step 2 Refinement
```

```
FUNCTION celsiusToFahrenheit(Celsius)
```

```
    RETURN (Celsius * 9 / 5) + 32
```

```
END FUNCTION
```

```
Step 3 Refinement
```

```
FUNCTION determineState(Celsius)
```

```
    IF Celsius <= 0 THEN
```

```
        PRINT "Solid"
```

```
    ELSE IF Celsius >= 100 THEN
```

```
        PRINT "Gas"
```

```
    ELSE
```

```
        PRINT "Liquid"
```

```
    END IF
```

```
END FUNCTION
```

```
Step 4 Refinement
```

```
FUNCTION findMax(Array, Size)
```

```
    DECLARE Max = Array[0]
```

```

    FOR i FROM 1 TO Size-1 DO
        IF Array[i] > Max THEN
            Max = Array[i]
        END IF
    Next i
    RETURN Max
END FUNCTION

```

Step 5 Refinement

```

FUNCTION findMin(Array, Size)
    DECLARE Min = Array[0]
    FOR i FROM 1 TO Size-1 DO
        IF Array[i] < Min THEN
            Min = Array[i]
        END IF
    END FOR
    RETURN Min
END FUNCTION

```

Step 6 Refinement

```

FUNCTION calculateAverage(Array, Size)
    DECLARE Sum = 0
    FOR i FROM 0 TO Size-1 DO
        Sum = Sum + Array[i]
    END FOR
    RETURN Sum / Size
END FUNCTION

```

Step 7 Refinement

```

FUNCTION countBelowThreshold(Array, Size, Threshold)

    DECLARE Count = 0

    FOR i FROM 0 TO Size-1 DO

        IF Array[i] < Threshold THEN

            Count = Count + 1

        END IF

    END FOR

    RETURN Count

END FUNCTION

```

Step 8 Refinement

```

FUNCTION displayresults(kelvin, celsius, Fahrenheit, SIZE)
    PRINT "Kelvin    Celsius    Fahrenheit    State"
    FOR i FROM 0 TO SIZE-1 DO
        PRINT KelvinTemp[i], "\t", CelsiusTemp[i], "\t",
FahrenheitTemp[i], "\t"
        CALL determineState(CelsiusTemps[i])
    END FOR

```

// Low-Level Algorithm: End